Buổi 2: Lập trình hướng đối tượng và biểu thức Lambda trong Kotlin (5 tiết)

Nội dung chính

2.1. Lập trình hướng đối tượng và biểu thức Lambda trong Kotlin

2.2. Tạo giao diện cho ứng dụng Android

Mục tiêu cần đạt được

Hiểu về lập trình hướng đối tượng trong Kotlin: Hiểu các khái niệm về lớp, đối tượng, hàm khởi tạo, kế thừa và cách chúng được sử dụng trong Kotlin.

Sử dụng biểu thức Lambda trong Kotlin: Biết cách sử dụng và truyền các biểu thức Lambda trong Kotlin để định nghĩa hàm ngắn gọn và linh hoạt.

Thiết kế giao diện Android cơ bản: Hiểu về cách tạo giao diện người dùng trong Android, sử dụng các View, Layouts, và xử lý sự kiện người dùng.

2.1. Lớp và đối tượng trong Kotlin

2.1.1. Lớp và đối tượng

Xác định một lớp

Định nghĩa lớp bắt đầu bằng từ khoá class, theo sau là tên và cặp dấu ngoặc nhọn. Phần cú pháp trước dấu ngoặc nhọn cũng được gọi là tiêu đề lớp. Trong dấu ngoặc nhọn, có thể chỉ định các thuộc tính và hàm cho lớp:

Hình 2.1: Định nghĩa lớp

Dưới đây là những quy ước đặt tên được đề xuất cho một lớp:

- Tên lớp không được dùng từ khóa Kotlin làm tên lớp, chẳng hạn như từ khóa fun.
- Tên lớp được viết bằng PascalCase, vì vậy, mỗi từ bắt đầu bằng một chữ cái viết hoa và không có dấu cách giữa các từ. Ví dụ: trong SmartDevice, chữ cái đầu tiên của mỗi từ được viết hoa và không có dấu cách giữa các từ.

Một lớp bao gồm 3 phần chính:

- Properties (Thuộc tính): Các biến chỉ định thuộc tính của đối tượng của lớp.
- Phương thức: Các hàm chứa hành vi và hành động của lớp.

2.1.2. Hàm khởi tạo

Hàm khởi tạo: Một hàm đặc biệt tạo các thực thể của lớp.

Ví dụ, tạo một lớp SmartDevice:

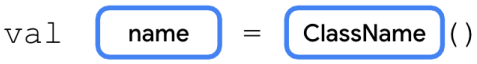
```
class SmartDevice {
    // empty body
}
fun main() {
}
```

Tạo một thực thể của lớp

Như chúng ta đã biết, lớp là bản thiết kế của một đối tượng. Để có thiết bị thông minh thực tế trong chương trình ta tạo một thực thể đối tượng SmartDevice. Cú pháp tạo thực thể bắt đầu bằng tên lớp, theo sau là một cặp dấu ngoặc đơn như có thể thấy trong hình dưới đây:

Hình 2.2: Tạo một thực thể cho lớp

Để sử dụng một đối tượng, ta sẽ tạo đối tượng và gán đối tượng đó cho một biến, tương tự như cách xác định một biến.



Hình 2.3: Tạo biến

Lưu ý: Khi xác định biến bằng từ khoá val để tham chiếu đối tượng, biến đó chỉ có thể đọc. Điều này có nghĩa là không thể chỉ định lại một đối tượng khác cho biến đó, nhưng có thể thay đổi trạng thái của đối tượng khi cập nhật các giá trị thuộc tính của biến.

Tạo thực thể lớp SmartDevice dưới dạng một đối tượng:

```
fun main() {
    val smartTvDevice = SmartDevice()
}
```

Xác định các phương thức của lớp

Các thao tác mà lớp có thể thực hiện được gọi là phương thức. Cú pháp khai báo phương thức trong lớp hoàn toàn tương tự như cú pháp tạo hàm thông thường. Sự khác biệt chính nằm ở việc phương thức được định nghĩa bên trong nội dung của lớp, từ đó nó trở thành một hàm thành phần, biểu thị cho hành vi của lớp.

Ví dụ, xác định phương thức turnOn() và turnOff() trong lớp SmartDevice:

```
class SmartDevice {
    fun turnOn() {
        println("Smart device is turned on.")
    }
```

```
    fun turnOff() {
        println("Smart device is turned off.")
    }
}
```

Gọi một phương thức trên một đối tượng

Lệnh gọi một phương thức trong một lớp tương tự như cách gọi các hàm khác từ hàm main().

```
class SmartDevice {
    fun turnOn() {
        turnOff()
        ...
    }
    ...
}
```

Để gọi một phương thức bên ngoài lớp, hãy bắt đầu bằng đối tượng lớp theo sau là toán tử ., tên hàm và một cặp dấu ngoặc đơn.

Hình 2.4: Gọi phương thức từ ngoài lớp

Gọi phương thức turnOn() và turnOff() trên đối tượng:

```
fun main() {
    val smartTvDevice = SmartDevice()
    smartTvDevice.turnOn()
}
```

Xác định các thuộc tính của lớp

Trong khi phương thức xác định các thao tác mà một lớp có thể thực hiện, thì các thuộc tính lại xác định những đặc điểm hoặc dữ liệu của lớp. Chẳng hạn, một thiết bị thông minh có thể có các thuộc tính như sau:

- Tên thiết bị: Xác định tên của thiết bị.
- Loại thiết bị: Xác định loại thiết bị thông minh, chẳng hạn như giải trí, tiện ích, hoặc nấu ăn.
- Trạng thái thiết bị: Cho biết thiết bị đang ở trạng thái bật, tắt, trực tuyến hay ngoại tuyến. Thiết bị được coi là trực tuyến khi kết nối với Internet, nếu không, nó sẽ được xem là ngoại tuyến.

Về cơ bản, thuộc tính là các biến được định nghĩa trong phần nội dung của lớp thay vì trong các phương thức. Cú pháp để khai báo thuộc tính và biến là tương tự nhau. Thuộc tính không thay đổi được xác định bằng từ khóa val, trong khi thuộc tính có thể thay đổi sử dụng từ khóa var. Ví dụ:

```
class SmartDevice {
```

```
    val name = "Android TV"
    val category = "Entertainment"
    var deviceStatus = "online"
    fun turnOn() {
        println("Smart device is turned on.")
    }
    fun turnOff() {
        println("Smart device is turned off.")
    }
}
```

Trên dòng sau biến smartTvDevice, hãy gọi hàm println() rồi truyền hàm này vào một chuỗi "Device name is: ${smartTvDevice.name}":

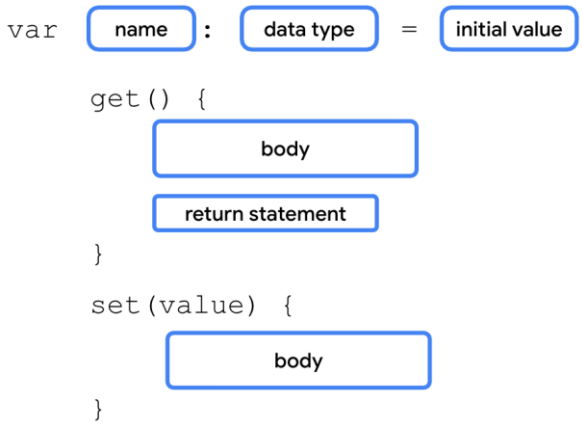
```
fun main() {
    val smartTvDevice = SmartDevice()
    println("Device name is: ${smartTvDevice.name}")
    smartTvDevice.turnOn()
    smartTvDevice.turnOff()
}
```

Chạy mã. Kết quả sẽ như sau:

```
Device name is: Android TV
Smart device is turned on.
Smart device is turned off.
Hàm getter và setter trong các thuộc tính
```

Thuộc tính có thể thực hiện nhiều hơn biến, ví dụ như giới hạn giá trị âm lượng TV từ 0 đến 100 bằng hàm setter hoặc tự động chuyển đổi tên thành chữ in hoa bằng hàm getter. Cú pháp khai báo thuộc tính có thể bao gồm các hàm get() và set() tùy chọn.

Cú pháp đầy đủ để khai báo thuộc tính có thể thay đổi bắt đầu bằng định nghĩa biến, kèm theo các hàm get() và set() (tùy chọn).



Hình 2.5: Hàm set() và get()

Khi không xác định hàm getter và setter cho một thuộc tính, trình biên dịch Kotlin sẽ tạo các hàm đó trong nội bộ. Ví dụ: nếu sử dụng từ khoá var để xác định một thuộc tính speakerVolume và gán một giá trị 2 cho thuộc tính đó, trình biên dịch sẽ tự động tạo các hàm getter và setter như có thể thấy trong đoạn mã dưới đây:

```
var speakerVolume = 2
    get() = field
    set(value) {
        field = value
    }
```

Ta sẽ không thấy những dòng này trong mã vì trình biên dịch đã thêm chúng vào nền.

Cú pháp đầy đủ cho thuộc tính không thể thay đổi có hai điểm khác biệt:

Nó bắt đầu với từ khóa val.

Các biến của val là các loại chỉ được đọc, do đó, chúng không có hàm set().

Thuộc tính Kotlin sử dụng trường sao lưu để lưu giữ giá trị trong bộ nhớ. Trường sao lưu về cơ bản là một biến lớp được xác định nội bộ bên trong các thuộc tính. Trường sao lưu nằm trong phạm vi một thuộc tính, có nghĩa là chỉ có thể truy cập vào trường đó thông qua các hàm thuộc tính get() hoặc set().

Để đọc giá trị thuộc tính trong hàm get() hoặc cập nhật giá trị trong hàm set(), cần sử dụng trường sao lưu của thuộc tính. Tệp này được trình biên dịch Kotlin tạo tự động và được tham chiếu bằng giá trị nhận dạng field.

Ví dụ: khi muốn cập nhật giá trị của thuộc tính trong hàm set(), sử dụng tham số của hàm set() (còn được gọi là tham số value) và gán giá trị đó cho field biến như có thể thấy trong đoạn mã này:

```
var speakerVolume = 2
    set(value) {
        field = value
    }
```

Cảnh báo: Không được sử dụng tên thuộc tính để nhận hoặc đặt giá trị. Ví dụ: trong hàm set(), nếu có ý định chỉ định tham số value cho chính thuộc tính speakerVolume, thì mã sẽ nhập vào một vòng lặp vô hạn vì thời gian chạy Kotlin cố gắng cập nhật giá trị cho speakerVolume thuộc tính này, nó sẽ kích hoạt lệnh gọi đến hàm setter nhiều lần.

Ví dụ: để đảm bảo giá trị được gán cho thuộc tính speakerVolume nằm trong khoảng từ 0 đến 100, có thể triển khai hàm setter như có thể thấy trong đoạn mã dưới đây:

```
var speakerVolume = 2
    set(value) {
        if (value in 0..100) {
            field = value
```

```
        }
    }
```

Xác định hàm khởi tạo

Mục đích chính của hàm khởi tạo là chỉ định cách các đối tượng của lớp được tạo. Nói cách khác, hàm khởi tạo sẽ tạo một đối tượng và khiến đối tượng đó sẵn sàng để sử dụng. Mã bên trong hàm khởi tạo thực thi khi đối tượng của lớp được tạo thực thể. Một hàm khởi tạo có hoặc không có tham số.

2.1.2. Hàm khởi tạo

Hàm khởi tạo mặc định là hàm khởi tạo không có tham số:

```
class SmartDevice constructor() {
    ...
}
```

Kotlin luôn hướng đến sự súc tích, vì vậy có thể xoá từ khoá constructor nếu không có chú thích trên hàm khởi tạo. Có thể xoá dấu ngoặc đơn nếu hàm khởi tạo không có tham số như minh hoạ trong đoạn mã dưới đây:

```
class SmartDevice {
    ...
}
```

Trình biên dịch Kotlin sẽ tự động tạo hàm khởi tạo mặc định. Ta sẽ không thấy hàm khởi tạo mặc định được tạo tự động trong mã vì hàm đó đã được trình biên dịch thêm vào trong nền.

Xác định một hàm khởi tạo có tham số

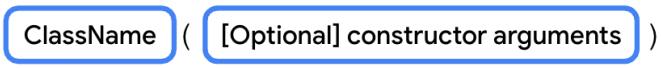
Để duy trì tính bất biến nhưng tránh các giá trị được mã hoá cứng, hãy dùng một hàm khởi tạo có tham số để tạo các giá trị đó:

```
class SmartDevice(val name: String, val category: String) {
    var deviceStatus = "online"
    fun turnOn() {
        println("Smart device is turned on.")
    }
    fun turnOff() {
        println("Smart device is turned off.")
    }
}
```

2.1.2. Hàm khởi tạo

Hàm khởi tạo hiện chấp nhận các tham số để thiết lập thuộc tính, vì vậy, cách tạo thực thể của một đối tượng cho lớp đó cũng sẽ thay đổi.

2.1.2. Hàm khởi tạo



Hình 2.6: Hàm khởi tạo chính

Lưu ý: Nếu lớp không có hàm khởi tạo mặc định và cố tạo thực thể cho đối tượng mà không có tham số, thì trình biên dịch sẽ báo cáo lỗi.

Ví dụ:

```
SmartDevice("Android TV", "Entertainment")
```

Cả hai tham số cho hàm khởi tạo đều là chuỗi. Nó không rõ ràng về giá trị cần chỉ định cho tham số. Để khắc phục lỗi này, tương tự như cách chuyển các tham số của hàm, có thể tạo một hàm khởi tạo có tham số được đặt tên như minh hoạ trong đoạn mã dưới đây:

```
SmartDevice(name = "Android TV", category = "Entertainment")
```

Có hai loại hàm khởi tạo chính trong Kotlin:

2.1.2. Hàm khởi tạo

Hàm khởi tạo chính. Một lớp chỉ có thể có một hàm khởi tạo chính, được xác định là một phần của tiêu đề lớp. Một hàm khởi tạo chính có thể là một hàm khởi tạo mặc định hoặc có tham số. Hàm khởi tạo chính không có nội dung. Điều đó có nghĩa là nó không được chứa bất kỳ mã nào.

Hàm khởi tạo phụ. Một lớp có thể có nhiều hàm khởi tạo phụ. Hàm khởi tạo phụ có hoặc không có tham số. Hàm khởi tạo phụ có thể khởi tạo lớp và có một phần nội dung có thể chứa logic khởi tạo. Nếu lớp có một hàm khởi tạo chính, mỗi hàm khởi tạo phụ đều phải tạo hàm khởi tạo chính.

Hình 2.7: Hàm khởi tạo phụ

Hàm khởi tạo phụ nằm trong phần nội dung của lớp và cú pháp của hàm này bao gồm 3 phần:

Khai báo hàm khởi tạo phụ. Việc xác định hàm khởi tạo phụ bắt đầu bằng từ khoá constructor theo sau là dấu ngoặc đơn. Nếu thích hợp, dấu ngoặc đơn chứa các tham số mà hàm khởi tạo phụ yêu cầu.

Khởi tạo hàm khởi tạo chính. Khởi tạo bằng dấu hai chấm, theo sau là từ khoá this và một cặp dấu ngoặc đơn. Nếu thích hợp, dấu ngoặc đơn sẽ chứa các tham số hàm khởi tạo chính yêu cầu.

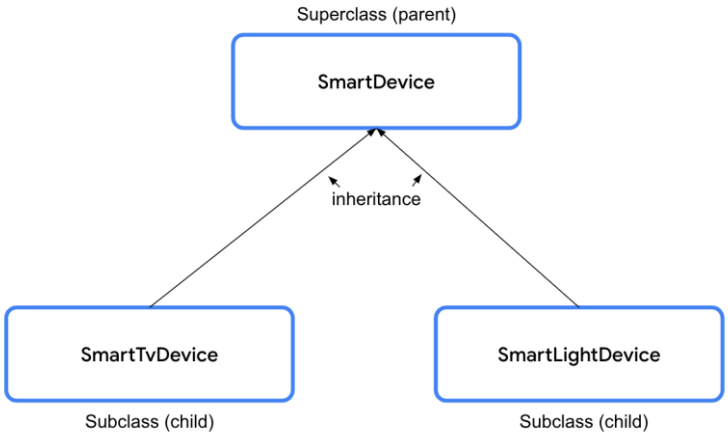
Nội dung hàm khởi tạo phụ. Khởi tạo hàm khởi tạo chính, theo sau là một cặp dấu ngoặc nhọn, chứa phần nội dung của hàm khởi tạo phụ.

Ví dụ:

```
class SmartDevice(val name: String, val category: String) {
    var deviceStatus = "online"
    constructor(name: String, category: String, statusCode:
Int) : this(name, category) {
        deviceStatus = when (statusCode) {
```

```
                0 -> "offline"
                1 -> "online"
                else -> "unknown"
            }
        }
        ...
    }
```

2.1.3. Kế thừa trong Kotlin



Hình 2.8: Kế thừa

Trong Kotlin, theo mặc định, tất cả các lớp đều là cuối cùng, nghĩa là không thể mở rộng chúng, vì vậy phải xác định mối quan hệ giữa các lớp.

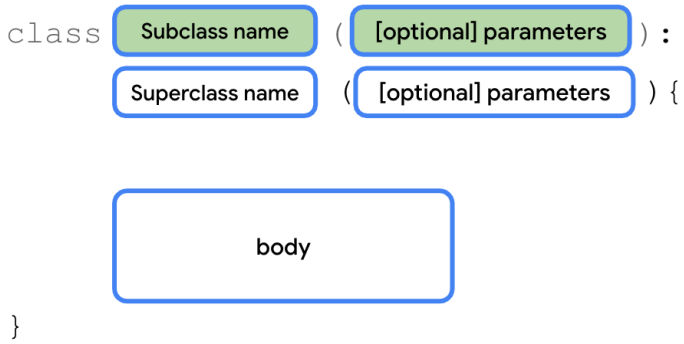
Xác định mối quan hệ giữa lớp cấp cao SmartDevice và các lớp con của chúng:

Trong lớp cấp cao SmartDevice, hãy thêm từ khoá open trước từ khoá class để có thể mở rộng từ khoá này:

```
open class SmartDevice(val name: String, val category: String)
{
    ...
}
```

Từ khoá open thông báo cho trình biên dịch rằng lớp này có thể mở rộng. Nhờ đó, vào lúc này lớp khác sẽ có thể mở rộng lớp này.

Cú pháp để tạo lớp con bắt đầu bằng việc tạo tiêu đề lớp như đã làm từ trước đến nay. Sau dấu ngoặc đơn đóng của hàm khởi tạo là dấu cách, dấu hai chấm, dấu cách khác, tên lớp cấp cao và một cặp dấu ngoặc đơn. Nếu cần, dấu ngoặc đơn sẽ bao gồm các tham số mà hàm khởi tạo lớp cấp cao yêu cầu. có thể xem cú pháp trong sơ đồ này:



Hình 2.9: Tạo lớp con

Tạo lớp con SmartTvDevice mở rộng lớp cấp cao SmartDevice:

class SmartTvDevice(deviceName: String, deviceCategory: String) :

```
    SmartDevice(name = deviceName, category = deviceCategory)
{
}
```

Việc xác định constructor cho SmartTvDevice không chỉ định xem các thuộc tính này có thể thay đổi hay không thể thay đổi. Điều này có nghĩa là tham số deviceName và deviceCategory chỉ là tham số constructor hơn là thuộc tính lớp.

Trong phần nội dung của lớp con SmartTvDevice, hãy thêm thuộc tính speakerVolume đã tạo khi được học về các hàm getter và setter:

```
class SmartTvDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }
}
```

Xác định một thuộc tính channelNumber được chỉ định cho giá trị 1 bằng một hàm setter chỉ định một phạm vi 0..200:

class SmartTvDevice(deviceName: String, deviceCategory: String) :

```
    SmartDevice(name = deviceName, category = deviceCategory)
{
    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }
    var channelNumber = 1
        set(value) {
            if (value in 0..200) {
                field = value
            }
```

```
        }
    }
```

Xác định phương thức increaseSpeakerVolume() tăng âm lượng và in chuỗi "Speaker volume increased to $speakerVolume.":

```
class SmartTvDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }
     var channelNumber = 1
        set(value) {
            if (value in 0..200) {
                field = value
            }
        }
    fun increaseSpeakerVolume() {
        speakerVolume++
        println("Speaker volume increased to $speakerVolume.")
    }
}
```

Ghi đè phương thức của lớp cấp cao từ các lớp con

Khi ghi đè một phương thức, phương thức trong lớp con sẽ làm gián đoạn quá trình thực thi phương thức được xác định trong lớp cấp cao và cung cấp phương thức thực thi của chính nó.

Ghi đè lớp SmartDevice phương thức turnOn() và turnOff():

Trong phần nội dung của lớp cấp cao SmartDevice trước từ khoá fun của mỗi phương thức, hãy thêm từ khoá open:

```
open class SmartDevice(val name: String, val category: String)
{
    var deviceStatus = "online"
    open fun turnOn() {
        // function body
    }
    open fun turnOff() {
        // function body
    }
}
```

Viết đè các phương thức trong lớp con:

```
class SmartTvDevice(deviceName: String, deviceCategory:
String) : SmartDevice(name = deviceName, category =
deviceCategory) {
    ...
    fun turnOn() {
        deviceStatus = "on"
        println(
            "$name is turned on. Speaker volume is set to
$speakerVolume and channel number is " +
```

```
                "set to $channelNumber."
        )
    }
}
class SmartTvDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }
    var channelNumber = 1
        set(value) {
            if (value in 0..200) {
                field = value
            }
        }
    fun increaseSpeakerVolume() {
        speakerVolume++
        println("Speaker volume increased to $speakerVolume.")
    }
    fun nextChannel() {
        channelNumber++
        println("Channel number increased to $channelNumber.")
    }
    override fun turnOn() {
        deviceStatus = "on"
        println(
            "$name is turned on. Speaker volume is set to
$speakerVolume
and channel number is " +
                "set to $channelNumber."
        )
    }
    override fun turnOff() {
        deviceStatus = "off"
        println("$name turned off")
    }
}
```

Trong hàm main():

```
fun main() {
    var smartDevice: SmartDevice = SmartTvDevice("Android TV",
"Entertainment")
    smartDevice.turnOn()
}
```

Cú pháp để gọi phương thức từ lớp cấp cao bắt đầu bằng từ khoá super, theo sau là toán tử ., tên hàm và một cặp dấu ngoặc đơn. Nếu có thể, dấu ngoặc đơn có bao gồm các tham số. có thể xem cú pháp trong sơ đồ này:

Hình 2.10: Gọi phương thức từ lớp cấp cao

Sử dụng lại mã trong lớp cấp cao SmartDevice:

Xoá các câu lệnh println() khỏi phương thức turnOn() và turnOff(), đồng thời di chuyển mã trùng lặp từ lớp con SmartTvDevice và SmartLightDevice sang lớp cấp cao SmartDevice:

```
open class SmartDevice(val name: String, val category: String)
{
    var deviceStatus = "online"
    open fun turnOn() {
        deviceStatus = "on"
    }
    open fun turnOff() {
        deviceStatus = "off"
    }
}
```

Dùng từ khoá super để gọi các phương thức từ lớp SmartDevice trong các lớp con SmartTvDevice và SmartLightDevice:

```
class SmartTvDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    var speakerVolume = 2
        set(value) {
            if (value in 0..100) {
                field = value
            }
        }
     var channelNumber = 1
        set(value) {
            if (value in 0..200) {
                field = value
            }
        }
    fun increaseSpeakerVolume() {
        speakerVolume++
        println("Speaker volume increased to $speakerVolume.")
    }
    fun nextChannel() {
        channelNumber++
        println("Channel number increased to $channelNumber.")
    }
    override fun turnOn() {
        super.turnOn()
        println(
            "$name is turned on. Speaker volume is set to
$speakerVolume and channel number is " +
                "set to $channelNumber."
        )
    }
    override fun turnOff() {
        super.turnOff()
        println("$name turned off")
    }
}
```

Ghi đè các thuộc tính lớp cấp cao từ lớp con

Tương tự như phương thức, cũng có thể ghi đè các thuộc tính theo các bước tương tự. Ghi đè thuộc tính deviceType:

Trong lớp cấp cao SmartDevice trên dòng sau thuộc tính deviceStatus, hãy dùng từ khoá open và val để xác định thuộc tính deviceType được đặt thành chuỗi "unknown":

```
open class SmartDevice(val name: String, val category: String)
{
    var deviceStatus = "online"
    open val deviceType = "unknown"
    ...
}
```

Trong lớp SmartTvDevice, hãy dùng từ khoá override và val để xác định thuộc tính deviceType được đặt thành chuỗi "Smart TV":

```
class SmartTvDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    override val deviceType = "Smart TV"
    ...
}
```

Trong lớp SmartLightDevice, hãy dùng từ khoá override và val để xác định thuộc tính deviceType được đặt thành chuỗi "Smart Light":

```
class SmartLightDevice(deviceName: String, deviceCategory:
String) :
    SmartDevice(name = deviceName, category = deviceCategory)
{
    override val deviceType = "Smart Light"
    ...
}
```

Phạm vi truy cập trong Kotlin:

Đối tượng sửa đổi chế độ hiển thị mặc định public: Cho phép khai báo có thể truy cập ở mọi nơi. Các thuộc tính và phương thức muốn dùng bên ngoài lớp sẽ được đánh dấu công khai.

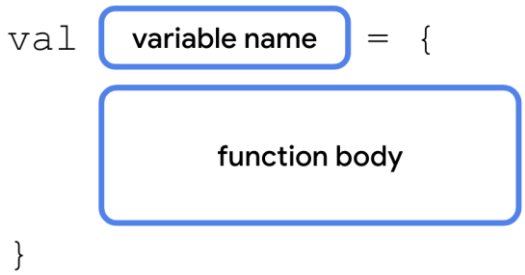
- Private: Cho phép khai báo có thể truy cập trong cùng một lớp hoặc tệp nguồn.
- Protected: Cho phép khai báo có thể truy cập trong các lớp con. Các thuộc tính và phương thức muốn dùng trong lớp xác định các lớp đó và lớp con được đánh dấu bằng đối tượng sửa đổi chế độ hiển thị protected.
- Internal: Cho phép khai báo có thể truy cập được trong cùng một mô-đun. Đối tượng sửa đổi nội bộ cũng giống như đối tượng sửa đổi riêng tư, nhưng có thể truy cập vào các thuộc tính và phương thức nội bộ từ bên ngoài lớp, miễn là đối tượng này được truy cập trong cùng một mô-đun.

2.3. Biểu thức Lambda

Hàm vừa là cấu trúc lớp thứ nhất vừa là loại dữ liệu, do đó ta có thể lưu trữ hàm trong biến, truyền chúng vào hàm và trả về chúng từ hàm. Tất cả những điều này có thể thực hiện được nhờ biểu thức lambda.

Biểu thức lambda cung cấp cú pháp ngắn gọn để xác định hàm không có từ khoá fun. Ta có thể lưu trữ biểu thức lambda trực tiếp trong biến mà không cần tham chiếu hàm trên hàm khác.

Trước toán tử chỉ định (=), thêm từ khoá val hoặc var, theo sau là tên biến (tên được sử dụng khi gọi hàm). Sau toán tử chỉ định (=) là biểu thức lambda, bao gồm một cặp dấu ngoặc nhọn tạo thành nội dung hàm. Cú pháp được minh hoạ trong hình ảnh này:

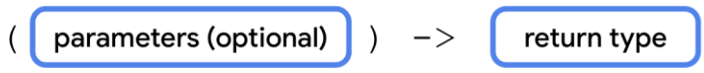


Hình 2.10: Biểu thức Lambda

Ví dụ:

```
fun main() {
    val trickFunction = trick
    trick()
    trickFunction()
}
val trick = {
    println("No treats!")
}
```

Kotlin có suy luận loại dữ liệu. Khi khai báo một biến không cần chỉ định rõ loại. Tuy nhiên, nếu muốn chỉ định loại tham số hàm hoặc loại dữ liệu trả về, ta cần biết cú pháp để thể hiện loại hàm. Loại hàm bao gồm một nhóm ngoặc đơn chứa danh sách tham số tuỳ chọn, ký hiệu -> và loại dữ liệu trả về. Cú pháp được minh hoạ trong hình ảnh này:



Hình 2.11 Chỉ định tham số và loại dữ liệu trả về

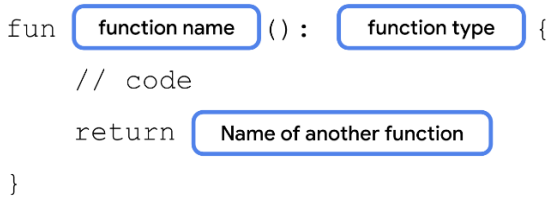
Nếu một hàm nhận hai tham số Int và trả về một Int, thì loại dữ liệu của hàm đó sẽ là (Int, Int) -> Int.

```
val treat: () -> Unit = {
    println("Have a treat!")
}
```

```
fun main() {
    treat()
}
```

Dùng hàm làm loại dữ liệu trả về

Hàm là loại dữ liệu, vì vậy có thể dùng hàm như bất kỳ loại dữ liệu nào khác. Thậm chí có thể trả về hàm từ hàm khác. Cú pháp được minh hoạ trong hình ảnh này:



Hình 2.12: Dùng hàm làm loại dữ liệu trả về

```
fun trickOrTreat(isTrick: Boolean, extraTreat: (Int) ->
String): () -> Unit {
    if (isTrick) {
        return trick
    } else {
        println(extraTreat(5))
        return treat
    }
}
fun main() {
    val coins: (Int) -> String = { quantity ->
        "$quantity quarters"
    }

    val cupcake: (Int) -> String = {
        "Have a cupcake!"
    }
    val treatFunction = trickOrTreat(false, coins)
    val trickFunction = trickOrTreat(true, cupcake)
    treatFunction()
    trickFunction()
}
```

Biểu thức lambda trong Kotlin có thể được viết gọn hơn bằng cách bỏ qua tên tham số và sử dụng từ khoá ngầm định "it" khi hàm chỉ có một tham số. cũng có thể truyền biểu thức lambda trực tiếp vào hàm mà không cần khai báo biến, và khi lambda là tham số cuối của hàm, nó có thể được đặt sau dấu ngoặc đơn đóng của hàm. Điều này giúp mã ngắn gọn và dễ đọc hơn mà không thay đổi chức năng của nó.

```
val coins: (Int) -> String = { quantity ->
    "$quantity quarters"
}
```

↓

```
val coins: (Int) -> String = {
    "$it quarters"
}
```

Hình 2.13: Ví dụ viết gọn hàm lambda

```
val coins: (Int) -> String = { "$it quarters" }
fun main() {
    val treatFunction = trickOrTreat(false) { "$it quarters" }
    val trickFunction = trickOrTreat(true, null)
    treatFunction()
    trickFunction()
}
```

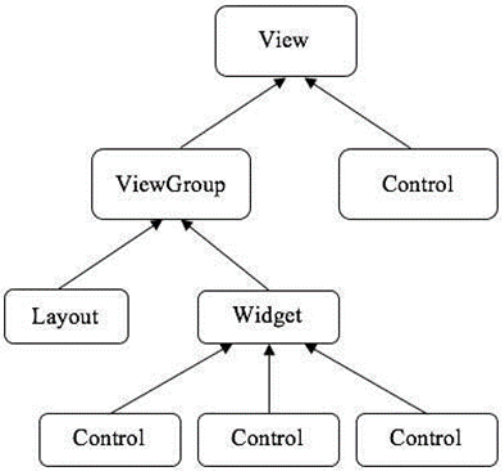
2.2. Tạo giao diện cho ứng dụng

2.2.1. Tạo giao diện với khung hiển thị - View của Android

View là lớp giao diện người dùng cơ bản nhất chứa các lớp giao diện như widget hay các lớp tổ chức giao diện (gọi là các layout). ViewGroup là lớp mở rộng của đối tượng View, một ViewGroup có thể chứa nhiều đối tượng View. Các đối tượng widgets trong một ViewGroup kết nối với nó thông qua các View con. Các ViewGroup cũng được mở rộng để chứa các thành phần layout của ứng dụng. Các đối tượng Activities giống như một cửa sổ (window) hay một Form chứa các thành phần điều khiển trên giao diện cho phép người dùng tương tác với ứng dụng Android. Mỗi Activity đều gắn liền với một đối tượng View mặc định. Android cho phép xây dựng giao diện ứng dụng bằng cách khai báo và khởi tạo các widgets và các layout phù hợp, nhưng thông thường các nhà phát triển ứng dụng thường định nghĩa giao diện ứng dụng trong một file XML. Điều này cho phép tách biệt riêng giữa phần giao diện và phần code tạo nên sự mềm dẻo và linh hoạt khi phát triển ứng dụng.

Các thành phần giao diện trong file XML được hiển thị trên giao diện người dùng thông qua đối tượng View. Để thiết lập nội dung cho đối tượng View, người lập trình cần sử dụng phương thức: setContentView(R.layout.tên_file_xml);

Các thành phần giao diện trong file XML được hiển thị trên giao diện người dùng thông qua đối tượng View. Để thiết lập nội dung cho đối tượng View, người lập trình cần sử dụng phương thức: setContentView(R.layout.tên_file_xml);



Hình 2.14: lớp View và các lớp con của nó

Giả sử file XML chứa các thành phần giao diện được hiển thị trên một View là main_activity.xml trong thư mục layout của ứng dụng thì câu lệnh trên viết như sau:

```
setContentView(R.layout.main_activity)
```

Các đối tượng widget trên giao diện người dùng trong một Activity được truy cập thông qua View bằng cách sử dụng phương thức findViewById(). Chẳng hạn, người lập trình lấy đối tượng EditText được định nghĩa trong file XML như sau:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.main_activity)
    val name = findViewById<EditText>(R.id.name)
}
```

Khi muốn kiểm tra xem đối tượng nào trên giao diện nào được chọn (click) thường sử dụng phương thức getId(). Ví dụ, với mỗi nút nhấn (Button) được chọn thì thực hiện các hành động tương ứng như sau:

```
override fun onClick(v: View) {
    when (v.id) {
        R.id.btn_play -> play() // button thứ nhất
        R.id.btn_pause -> pause() // button thứ hai
        R.id.btn_playpre -> playPre() // button thứ ba
        R.id.btn_playnext -> playNext() // button thứ tư
        R.id.btn_playlist -> showPlaylist(mSeekBar) // button
thứ năm
    }
}
```

2.2.2. Layout trong giao diện Android

Các đối tượng Layout được sử dụng để tổ chức các đối tượng widget trong giao điện theo các cách khác nhau (thường gọi là các containers). Một file XML layout thường sử dụng một trong các đối tượng Layout làm đối tượng cha của các đối tượng widget và layout khác trong giao diện. Android có nhiều đối tượng layout là FrameLayout, LinearLayout, RelativeLayout, TableLayout và AbsoluteLayout …

Đối tượng FrameLayout cho phép giới hạn vị trí một đối tượng trên màn hình làm cho giao diện ứng dụng vẫn giữ được tổ chức khi kích thước màn hình thay đổi. Đối tượng LinearLayout cho phép tổ chức các đối tượng widget con của nó lần lượt theo chiều dọc hoặc theo chiều ngang, đồng thời chúng được căn theo lề trái, lề phải hoặc căn giữa,. . . . Phần dưới đây sẽ trình bày cách sử dụng một số đối tượng layout phổ biến trong Android.

Đối tượng FrameLayout

Như đã giới thiệu ở phần trên, đối tượng FrameLayout giới hạn phạm vi của một đối tượng trên giao diện khi kích thước màn hình thay đổi. Một số thuộc tính cơ bản đối tượng FrameLayout gồm:

- id: định danh của đối tượng
- foreground: màu sắc của đối tượng khi đưa con trỏ qua
- foregroundGravity: căn các đối tượng con, có thể nhận các giá trị như top, bottom, left, right, center, cent_vertical, center_horizontal,. . . .

Ví dụ: Tạo một file XML layout activ- ity_main.xml với đối tượng FrameLayout như sau:

```
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <ImageView
        android:src="@drawable/ic_launcher"
        android:scaleType="fitCenter"
        android:layout_width="250dp"
        android:layout_height="250dp" />
    <TextView
        android:text="Frame Demo"
        android:textSize="30sp"
        android:textStyle="bold"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
Trong file res/values/strings.xml định nghĩa tên ứng dụng:
<string name="app_name">FrameLayout Demo</string>
```

Đối tượng LinearLayout

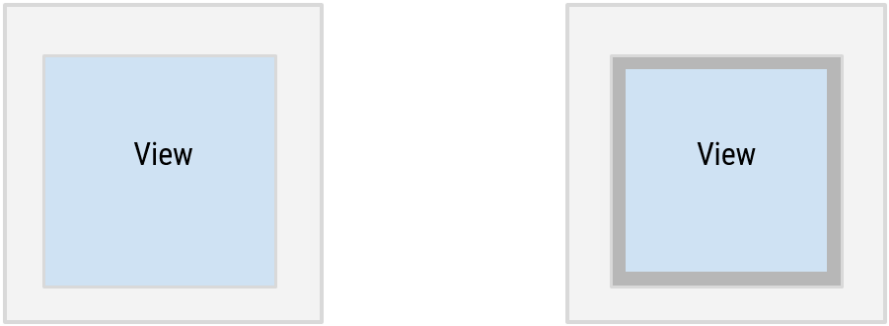
LinearLayout là một mô hình cho phép sắp xếp các đối tượng widget hoặc các container con theo dòng hoặc theo cột. Dưới đây sẽ trình bày các khái niệm cơ bản và thuộc tính của đối tượng LinearLayout.

Orientation: xác định các đối tượng con trong LinearLayout được trình bày theo dòng hoặc theo cột và được chỉ định qua thuộc tính android:orientation của nó trong file XML. Trong chương trình java có thể thay đổi giá trị thuộc tính này thông qua phương thức setOrientation() với 2 giá trị là HORIZONTAL và VERTICAL.

Fill Model: Tất cả các đối tượng widget đều phải được cung cấp thuộc tính chiều rộng (android:layout_width) và chiều cao (an- droid:layout_height), chúng có thể nhận một trong các giá trị sau:

- Chỉ định giá trị cụ thể theo pixel (ví du: 125px)

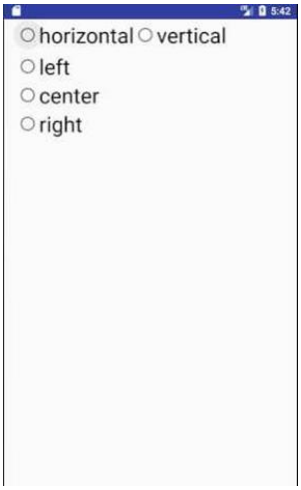
- wrap_content: đối tượng sẽ điền đầy không gian tự nhiên còn trống trên màn hình.
- fill_parent: đối tượng sẽ được điền kín trong không gian của đối tượng cha (một container) chứa nó.
- Weight: chỉ khoảng cách giữa các đối tượng widget con và được thiết lập thông qua thuộc tính android:layout_weight.
- Gravity: cách căn các đối tượng con theo lề trái, lề phải, ở giữ hay kết hợp giữa các kiểu, nó được xác định qua thuộc tính android:layout_gravity hoặc sử dụng phương thức setGravity(). Khi căn một cột nó thường nhận một trong các giá trị: left, cen- ter_horizontal , right, khi căn một dòng nó thường nhận giá trị center_vertical.
- Padding: Cho phép thiết lập khoảng trống giữa các đối tượng widget qua thuộc tính android:padding hoặc sử dụng phương thức setPadding(). Thuộc tính này cho phép thiết lập khoảng trống cả bốn phía, để thiết lập riêng theo từng hướng có thể sử dụng các thuộc tính android:paddingLeft, android:paddingRight, an- droid:paddingTop và android:paddingBottom.



Hình 2.15: Mình hoạ thuộc tính Padding của đối tượng widget

Ví dụ: thiết kế giao diện sử dụng đối tượng LinearLayout chứa năm đối tượng Radio Button được nhóm thành hai nhóm như hình trên.

RadioGroup thứ nhất chứa hai Radio Button có nhãn horizontal và vertical, RadioGroup thứ hai chứa ba RadioButton là left, center và right. Khi chọn Radio Button có nhãn nào thì cho phép sắp xếp hoặc căn các đối tượng widget trong nhóm theo chức năng tương ứng.



Hình 2.16: Ví dụ đối tượng LinearLayout

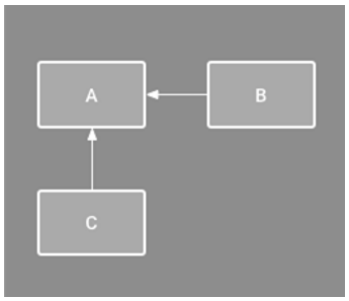
Phương thức onCreate() chứa đoạn mã như sau:

```
orientation = findViewById<RadioGroup>(R.id.orientation)
orientation.setOnCheckedChangeListener {
    // Xử lý khi orientation thay đổi
}
gravity = findViewById<RadioGroup>(R.id.gravity)
gravity.setOnCheckedChangeListener {
    // Xử lý khi gravity thay đổi
}
Phương thức viết đè onCheckedChanged chứa nội dung như sau:
when (checkedId) {
    R.id.horizontal -> {
        // Sắp xếp các con theo chiều ngang
        orientation.orientation = LinearLayout.HORIZONTAL
    }
    R.id.vertical -> {
        // Sắp xếp các con theo chiều dọc
        orientation.orientation = LinearLayout.VERTICAL
    }
    R.id.left -> {
        // Căn các con theo lề trái của cột
        gravity.gravity = Gravity.LEFT
    }
    R.id.center -> {
        // Căn các con giữa cột
        gravity.gravity = Gravity.CENTER_HORIZONTAL
    }
    R.id.right -> {
        // Căn các con theo lề phải của cột
        gravity.gravity = Gravity.RIGHT
    }
}
```

ConstraintLayout

ConstraintLayout là một loại layout được khuyên dùng cho Android, giúp giải quyết vấn đề về các layout lồng nhau quá nhiều, gây chậm và không phản hồi kịp thời cho UI. Với ConstraintLayout, người lập trình thể định vị và kích thước các view bằng cách sử dụng một tập hợp các ràng buộc (constraints).

Định nghĩa Ràng Buộc (Constraint):



Hình 2.17: Ràng buộc

Một ràng buộc là sự hạn chế hoặc giới hạn trên các thuộc tính của một view mà layout cố gắng tuân thủ. Ví dụ: view B có thể được ràng buộc luôn nằm bên phải của view A, và view C được ràng buộc nằm bên dưới của view A.

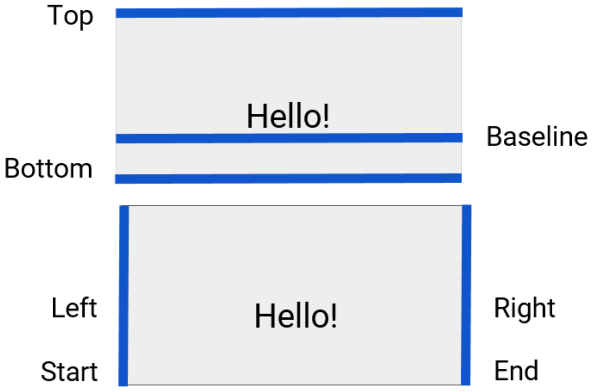
Các loại ràng buộc phổ biến:

Ràng buộc tương đối với container cha: Có thể thiết lập ràng buộc dựa trên các cạnh của container cha. Ví dụ:

app:layout_constraintTop_toTopOf="parent": Ràng buộc cạnh trên của view với cạnh trên của container cha.

app:layout_constraintStart_toStartOf="parent": Ràng buộc cạnh trái của view với cạnh trái của container cha.

Ràng buộc theo vị trí tương đối: có thể thêm ràng buộc theo hướng ngang (start, end) và hướng dọc (top, bottom, baseline) cho các view.



Hình 2.18: Các vị trí tương đối

Ví dụ để có thể căn một dòng chữ vào giữa màn hình ta sẽ làm như sau:

Hình 2.19: Tạo TextView nằm giữa màn hình

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Các khái niệm mở rộng:

Guideline: Cho phép định vị nhiều view theo một hướng dẫn cụ thể mà không bị vẽ trên thiết bị. Các guideline có thể là dọc hoặc ngang.

Chain: Cho phép định vị các view so với nhau theo cả chiều ngang và chiều dọc. Ví dụ, có thể tạo các chain với các style như Spread Chain, Packed Chain để điều chỉnh không gian giữa các view.

Bài tập bổ trợ

Bài tập thực hành cơ bản

1. Tạo lớp SmartDevice với các thuộc tính name, category và các phương thức như turnOn() và turnOff().
2. Viết chương trình sử dụng biểu thức Lambda để lọc danh sách các số nguyên chẵn từ 1 đến 10.
3. Tạo lớp Car với các thuộc tính brand, model, price. Thêm phương thức hiển thị thông tin xe.
4. Viết chương trình sử dụng biểu thức Lambda để lọc danh sách các chuỗi có độ dài lớn hơn 5.
5. Sử dụng kế thừa để tạo lớp con ElectricCar kế thừa từ lớp Car và thêm thuộc tính batteryCapacity.
6. Sử dụng biểu thức Lambda để sắp xếp danh sách số nguyên giảm dần.
7. Mục tiêu: Tạo giao diện đăng nhập đơn giản: Sử dụng LinearLayout theo hướng dọc, có hai EditText cho tên người dùng và mật khẩu, nút Button với dòng chữ "Đăng nhập".
8. Tạo giao diện ứng dụng máy tính cơ bản: Sử dụng GridLayout hoặc TableLayout, tạo các nút số từ 0-9 và các nút phép tính: +, -, *, /, =, kết quả hiển thị trong TextView phía trên.
9. Mục tiêu: Tạo giao diện hồ sơ cá nhân với ảnh đại diện: Sử dụng RelativeLayout hoặc ConstraintLayout, hiển thị ảnh đại diện với ImageView, hiển thị tên và mô tả ngắn gọn bằng TextView.

Bài giải mẫu

Bài 1: Tính toán số học

Viết một hàm calculate nhận ba tham số: hai số nguyên và một hàm lambda để thực hiện phép tính. Hãy tạo các lambda để:

- Cộng hai số.
- Trừ hai số.
- Chia hai số.

Gợi ý:

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int):
Int {
    return operation(a, b)
}

fun main() {
    val add = { x: Int, y: Int -> x + y }
    val subtract = { x: Int, y: Int -> x - y }
    val divide = { x: Int, y: Int -> x / y }

    println("Tổng: " + calculate(10, 5, add))
    println("Hiệu: " + calculate(10, 5, subtract))
```

```
    println("Thương: " + calculate(10, 5, divide))
}
```

Bài 2: Kiểm tra số chẵn/lẻ

Viết một hàm checkNumber nhận một số nguyên và một hàm lambda kiểm tra xem số đó có thỏa mãn điều kiện không.

Gợi ý:

```
fun checkNumber(num: Int, condition: (Int) -> Boolean):
Boolean {
    return condition(num)
}

fun main() {
    val isEven = { x: Int -> x % 2 == 0 }
    val isOdd = { x: Int -> x % 2 != 0 }

    println("Số 4 là số chẵn: " + checkNumber(4, isEven))
    println("Số 7 là số lẻ: " + checkNumber(7, isOdd))
}
```

Bài 3: Chuyển đổi chuỗi

Viết một hàm transformString nhận một chuỗi và một hàm lambda để chuyển đổi chuỗi đó. Tạo các lambda để:

- Viết hoa chuỗi.
- Viết thường chuỗi.
- Đảo ngược chuỗi.

Gợi ý:

```
fun transformString(input: String, action: (String) ->
String): String {
    return action(input)
}

fun main() {
    val toUpper = { str: String -> str.uppercase() }
    val toLower = { str: String -> str.lowercase() }
    val reverse = { str: String -> str.reversed() }

    println(transformString("Kotlin", toUpper))
    println(transformString("Kotlin", toLower))
    println(transformString("Kotlin", reverse))
}
```

Bài tập 4: Kiểm tra tuổi hợp lệ

Viết một hàm checkAge nhận một số nguyên và một lambda để kiểm tra xem số đó có nằm trong khoảng tuổi hợp lệ (18-60) hay không.

Gợi ý:

```
fun checkAge(age: Int, isValid: (Int) -> Boolean): Boolean {
    return isValid(age)
}

fun main() {
    val isAdult = { age: Int -> age in 18..60 }

    println("Tuổi 25 hợp lệ: " + checkAge(25, isAdult))
```

```
    println("Tuổi 65 hợp lệ: " + checkAge(65, isAdult))
}
```